Conservative default settings. The development team is aware of the attack surface for the product and minimizes it in the default configuration.

Avoidance of risky default changes. Applications do not make any default changes to the operating system or security settings that reduce security for the host computer. In some cases, such as for security products, it is acceptable for a software program to strengthen (increase) security settings for the host computer. The most common violations of this principle are games that either open firewall ports without informing the user or instruct users to open firewall ports without informing users of possible risks.

Less commonly used services off by default. If fewer than 80 percent of a program's users use a feature, that feature should not be activated by default. Measuring 80 percent usage in a product is often difficult because programs are designed for many different personas. It can be useful to consider whether a feature addresses a core/primary use scenario for all personas. If it does, the feature is sometimes referred to as a P1 feature.

Secure in Deployment

Deployment guides. Prescriptive deployment guides outline how to deploy each feature of a program securely, including providing users with information that enables them to assess the security risk of activating non-default options (and thereby increasing the attack surface).

Analysis and management tools. Security analysis and management tools enable administrators to determine and configure the optimal security level for a software release.

Patch deployment tools. Deployment tools aid in patch deployment.

Communications

Security response. Development teams respond promptly to reports of security vulnerabilities and communicate information about security updates.

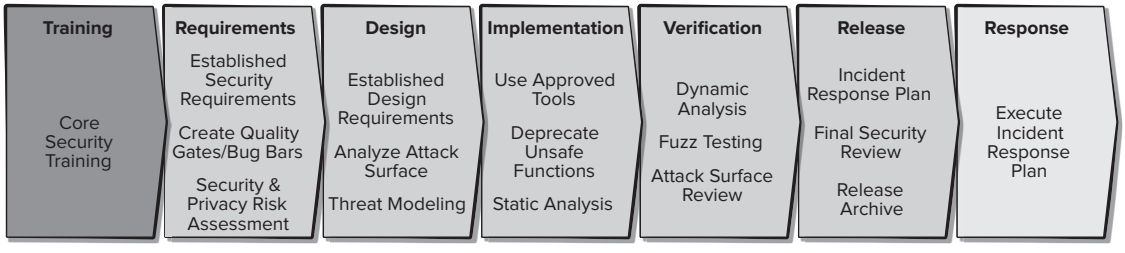
Community engagement. Development teams proactively engage with users to answer questions about security vulnerabilities, security updates, or changes in the security landscape.

The secure software development process model looks like the one shown in Figure 18.1.

The Microsoft SDL documentation describes what architects, designers, developers, and testers are required to do for each of the 16 recommended practices. The data that Microsoft collected after implementing the SDL shows a significant reduction in

FIGURE 18.1 **Secure Software Development Process Model at Microsoft**

Adapted from Shunn, A., et al. Strengths in Security Solutions, Software Engineering Institute, Carnegie Mellon University, 2013. Available at http://resources.sei.cmu.edu/library/asset-view.cfm?assetid=77878.



vulnerabilities, which led to a need for fewer patches, and thus a significant cost savings. We recommend that you browse the SDL website to learn more about these practices. Since the SDL was developed, there have been numerous papers, books, training, and so on, to go with the SDL model.[1]

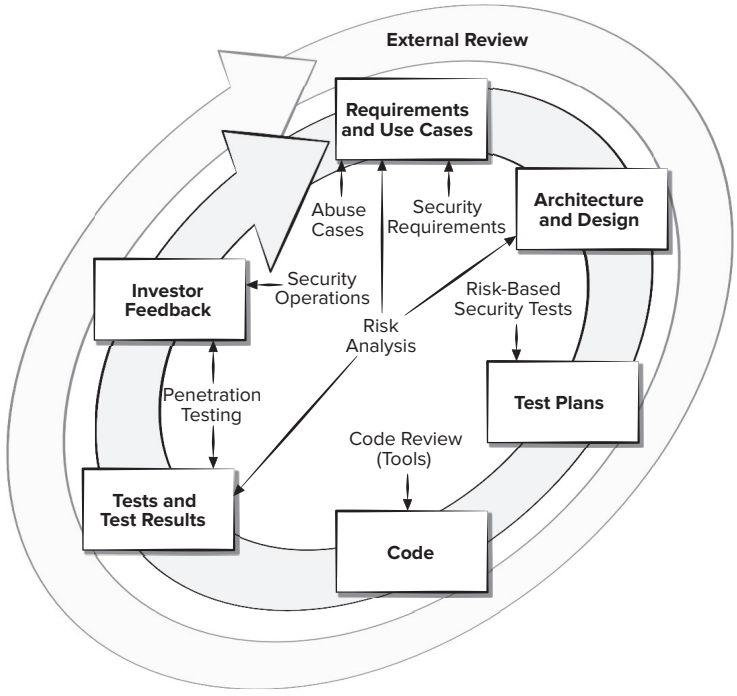
18.3 Secure Development Life-Cycle Activities

A different approach that is independent of a life-cycle model is the touchpoints for software security [McG06], which argues that the *activities* (touchpoints) are what matter, not the model. The activities can be incorporated into any life-cycle model, and thus are considered to be process agnostic. The touchpoints later formed the basis for BSIMM, a maturity model that will be discussed later in this chapter. Some organizations consider the touchpoints to be the minimum set of activities that should be performed in secure software development. A pictorial version of the touchpoints is shown in Figure 18.2. In this diagram, the recommended security activities appear above the corresponding software development activity, or life-cycle phase:

With the SDL and touchpoints in mind, we'll look at some of the important secure software development activities that are associated with them.

FIGURE 18.2

Software security touchpoints



1 More recently, Microsoft has shown how the SDL activities can be integrated with an agile development approach: https://www.microsoft.com/en-us/SDL/Discover/sdlagile.aspx.

18.4 SECURITY REQUIREMENTS ENGINEERING

Although security requirements are an important part of secure software development, in practice they are often neglected. When they exist, they are often an add-on, copied from a generic list of security features. The requirements engineering that is needed to get a better set of security requirements seldom takes place [All08].

Requirements engineering practice typically addresses desired user features. Therefore, attention is given to the functionality of the system from the user's perspective, but little attention is given to what the system should *not* do [Bis02]. Users expect systems to be secure, and these assumptions need to make their way into security requirements for software systems before they are developed, not after the fact. Often the users' assumptions about security are overlooked because system features are the primary focus.

In addition to security life-cycle models, there are many process models that are specific to security requirements. These include: core security requirements artifacts [Mof04], Software Cost Reduction (SCR) [Hei02], SQUARE (Security QUAlity Requirements Engineering) [Mea05], and Security Requirements Engineering Process (SREP) [Mel06]. For the remainder of this section, we'll consider SQUARE as a representative example of security life-cycle models.

18.4.1 SQUARE

SQUARE is a representative security requirements engineering process model, but it's important to keep in mind that if you already have a development process model, such as the one presented in Chapter 4, you can just pick up some of the SQUARE steps to enhance your existing model. There's no need to develop a whole new process to address security in your software development activities. We suggest that you add security definitions to your glossary; perform risk analysis, including identification of potential attacks via misuse cases or threat modeling; develop mitigation strategies; and perform categorization and prioritization of candidate security requirements.

The SQUARE process model provides for eliciting, categorizing, and prioritizing security requirements for software-intensive systems. Its focus is to build security concepts into the early stages of the development life cycle. It can also be used for fielded systems and those undergoing improvements and modifications. The process is shown in Table 18.1, followed by brief descriptions of each step.

18.4.2 The SQUARE Process

The SQUARE process is best applied by the project's requirements engineers and security experts with supportive executive management and stakeholders. Let's take a look at the steps.[2]

> **Step 1. Agree on definitions.** So that there is not semantic confusion, this step is needed as a prerequisite to security requirements engineering. On a

2 To dig deeper, see the resources available at the SEPA 9/e website.

TABLE 18.1 The SQUARE process

Number	Step	Input	Techniques	Participants	Output
1	Agree on definitions.	Candidate definitions from IEEE and other standards	Structured interviews, focus group	Stakeholders, requirements team	Agreed-to definitions
2	Identify assets and security goals.	Definitions, candidate assets and goals, business drivers, policies and procedures, examples	Facilitated work session, surveys, interviews	Stakeholders, requirements engineer	Assets and security goals
3	Develop artifacts to support security requirements definition.	Potential artifacts (e.g., scenarios, misuse cases, templates, forms)	Work session	Requirements engineer	Needed artifacts: scenarios, misuse cases, models, templates, forms
4	Perform (security) risk assessment.	Misuse cases, scenarios, security goals	Risk assessment method, analysis of anticipated risk against organizational risk tolerance, including threat analysis	Requirements engineer, risk expert, stakeholders	Risk assessment results
5	Select elicitation techniques.	Goals, definitions, candidate techniques, expertise of stakeholders, organizational style, culture, level of security needed, cost-benefit analysis, etc.	Work session	Requirements engineer	Selected elicitation techniques
6	Elicit security requirements.	Artifacts, risk assessment results, selected techniques	Accelerated Requirements Method, Joint Application Development, interviews, surveys, model-based analysis, checklists, lists of reusable requirements types, document reviews	Stakeholders facilitated by requirements engineer	Initial cut at security requirements
7	Categorize requirements as to level (system, software, etc.) and whether they are requirements or other kinds of constraints.	Initial requirements, architecture	Work session using a standard set of categories	Requirements engineer, other specialists as needed	Categorized requirements
8	Prioritize requirements.	Categorized requirements and risk assessment results	Prioritization methods such as Analytical Hierarchy Process (AHP), Triage, Win-Win, etc.	Stakeholders facilitated by requirements engineer	Prioritized requirements
9	Inspect requirements.	Prioritized requirements, candidate formal inspection technique	Inspection method such as Fagan and peer reviews	Inspection team	Initial selected requirements, documentation of decision-making process and rationale

given project, team members tend to have definitions in mind, based on their prior experience, but those definitions are often different from one another [Woo05]. Sources such as the Institute for Electrical and Electronics Engineers (IEEE) and the Software Engineering Body of Knowledge (SWEBOK) provide a range of definitions to select from or tailor [SWE14].

Step 2. Identify assets and security goals. This step occurs at the project's organizational level and is needed to support software development. Different stakeholders usually have concerns about different assets, and thus have different goals. For example, a stakeholder in human resources may be concerned about maintaining the confidentiality of personnel records, whereas a stakeholder in a research area may be concerned with ensuring that research project information is not accessed, modified, or stolen.

Step 3. Develop artifacts. This step is necessary to support all subsequent security requirements engineering activities. Often, organizations do not have key documents needed to support requirements definition, or they may not be up to date. This means that a lot of time may be spent backtracking to try to obtain documents, or the team will have to bring them up to date before going further.

Step 4. Perform risk assessment. This step requires an expert in risk assessment methods, the support of the stakeholders, and the support of a security requirements engineer. There are a number of risk assessment methods, but regardless of the one that you choose, the outcomes of risk assessment can help in identifying the high-priority security exposures.

Step 5. Select elicitation technique. This step becomes important when there are diverse stakeholders. A more formal elicitation technique, such as the Accelerated Requirements Method [Hub99], Joint Application Design [Woo89], or structured interviews, can be effective in overcoming communication issues when there are stakeholders with different cultural backgrounds. In other cases, elicitation may simply consist of sitting down with a primary stakeholder to try to understand that stakeholder's security requirements needs.

Step 6. Elicit security requirements. This step encompasses the actual elicitation process using the selected technique. Most elicitation techniques provide detailed guidance on how to perform elicitation. This builds on the artifacts that were developed in earlier steps.

Step 7. Categorize requirements. This step allows the security requirements engineer to distinguish among essential requirements, goals (desired requirements), and architectural constraints that may be present. This categorization also helps in the prioritization activity that follows.

Step 8. Prioritize requirements. This step depends on the prior step and may also involve performing a cost-benefit analysis to determine which security requirements have a high payoff relative to their cost. Of course prioritization may also depend on other consequences of security breaches, such as loss of life, loss of reputation, and loss of consumer confidence.

Step 9. Requirements inspection. This review activity can be accomplished at varying levels of formality, discussed in Chapter 16. Once inspection is complete, the project team should have an initial set of prioritized security requirements that can be revisited as needed later in the project.

18.5 Misuse and Abuse Cases and Attack Patterns

Misuse (or *abuse*) cases can help you view your software in the same way that attackers do. By thinking about negative events, you can better understand how to develop secure software. A misuse case can be thought of as a use case that the attacker initiates.

One of the goals of misuse cases [Sin00] is to decide up front how software should react to potential attacks. You can also use misuse and normal use cases together to conduct threat and hazard analysis [Ale03].

We suggest creating misuse cases through brainstorming. Teaming security experts with subject matter experts (SMEs) covers a lot of ground quickly. During brainstorming, software security experts ask many questions of developers to help identify the places where the system is likely to have weaknesses. This involves a careful look at all user interfaces and considers events that developers assume can't happen, but that attackers can actually cause to happen.

Here are some questions that need to be considered: How can the system distinguish between valid and invalid input data? Can it tell whether a request is coming from a legitimate application or a rogue application? Can an insider cause a system to malfunction? Trying to answer such questions helps developers to analyze their assumptions and allows them to fix problems up front.

Misuse cases can be in table or diagram form. Figure 18.3 provides an example misuse case that shows how DroidCleaner malware can successfully attack a cell phone using an open-source e-mail application called K-9. This is extracted from a much larger report that you may wish to study [Ali14].

In this misuse case, the user keeps e-mail on the phone's external storage area. The attacker gains access to the phone's storage by compromising the operating system. A common way for the attacker to gain access to the phone is by tricking the user into installing a Trojan, to which the user unwittingly grants access to the drive during the install process. The attacker is then able to use the Trojan to download files, including the e-mail contents file.

Attack patterns can provide some help by providing a blueprint for creating an attack. For example, buffer overflow is one type of security exploitation. Attackers trying to capitalize on a buffer overflow make use of similar steps [OWA16]. Attack patterns can document these steps (e.g., timing, resources, techniques) as well as practices software developers can use to prevent or mitigate their success [Hog04]. When you're trying to develop misuse and abuse cases, attack patterns can help.

Misuse cases need to be prioritized as they are generated. In addition, they need to strike the right balance between cost and benefit. The project budget may not allow